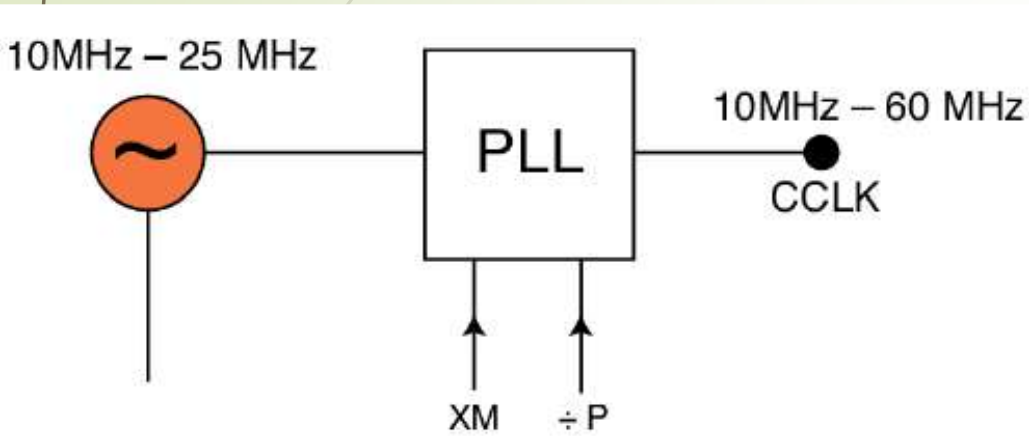
UART in LPC2148

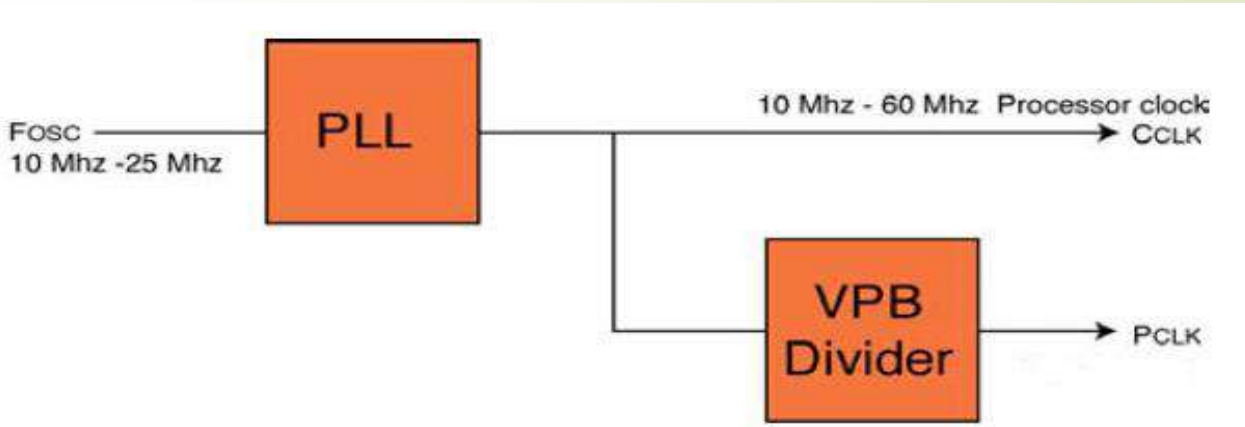


UART

- Two Identical UARTs → UART0 & UART1
- We are using UART0 for our example.

PLL



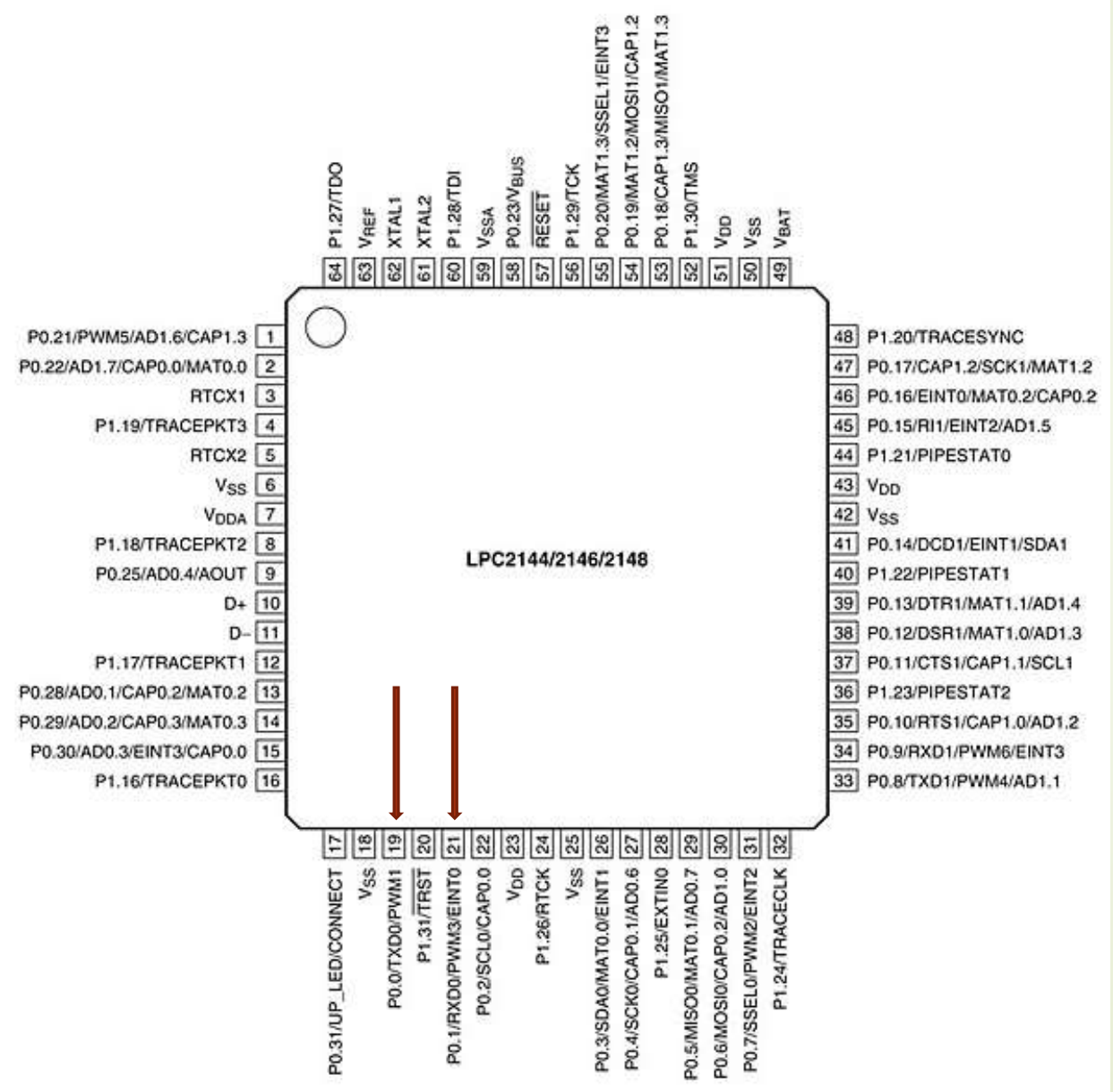


VPBDIV = 0x00000001;

Bit	Symbol	Value	Description	Reset value
1:0	VPBDIV	00	VPB bus clock is one fourth of the processor clock.	00
		01	VPB bus clock is the same as the processor clock.	
		10	VPB bus clock is one half of the processor clock.	
		11	Reserved. If this value is written to the VPBDIV register, it has no effect (the previous setting is retained).	

Pins Involved

P0.0 [Pin 19] ➡ TXD0
P0.1 [Pin 21] ➡ RXD0



SFRs Involved

- P0 Pin Select Register -PINSEL0
- UART0 Line Control Register -U0LCR
- UART0 Line Status Register -U0LSR
- UART0 Divisor Latch MSB -U0DLM
- UART0 Divisor Latch LSB -U0DLL
- UART0 Transmit Holding Register -U0THR
- UART0 Receive Buffer Register -U0RBR

Pin Select SFRs

- PINSEL0 for GPIO pins 0.0 to 0.15
- PINSEL1 for GPIO pins 0.16 to 0.31
- PINSEL2 for GPIO pins 1.16 to 1.31

Each Pin is Associated with multiple functions

Pin Select enable you to select which pin functions you would like to use

How to Use PINSEL

- Each associated 'pin' in PINSEL0 and PINSEL1 is assigned a 2-bit address.

PINSEL0

Pin	0.15	0.14	0.13	0.12	0.11	0.10	0.9	0.8	0.7	0.6	0.5	0.4	0.3	0.2	0.1	0.0
Bits	31..30	29..28	27..26	25..24	23..22	21..20	19..18	17..16	15..14	13..12	11..10	9..8	7..6	5..4	3..2	1..0

PINSEL1

Pin	0.31	0.30	0.29	0.28	0.27	0.26	0.25	0.24	0.23	0.22	0.21	0.20	0.19	0.18	0.17	0.16
Bits	31..30	29..28	27..26	25..24	23..22	21..20	19..18	17..16	15..14	13..12	11..10	9..8	7..6	5..4	3..2	1..0

Function Selection Bits

Function Selection Bits

Binary Value	Selected Function
00	Primary (default) function (always GPIO)
01	First alternate function
10	Second alternate function
11	Third alternate function

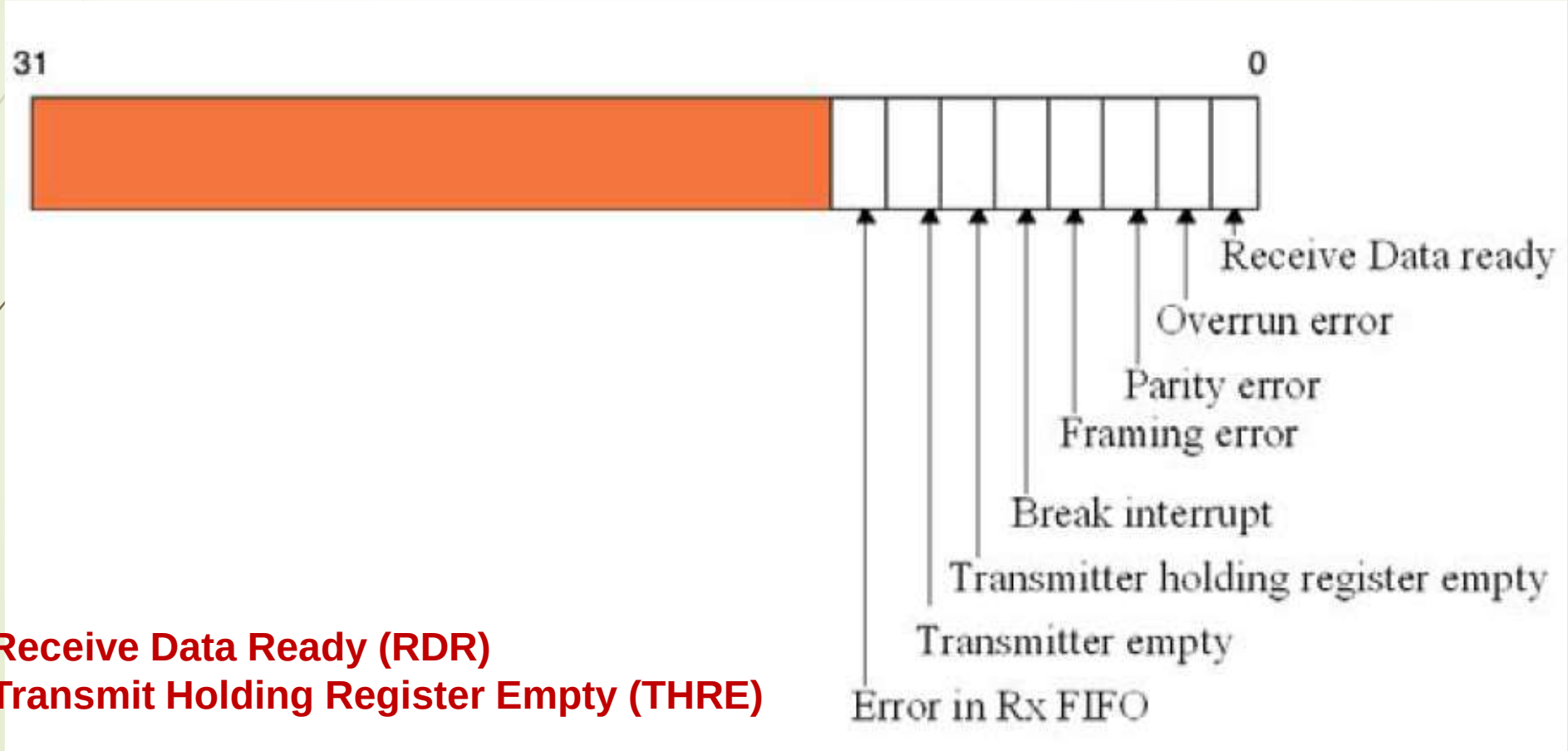
Example for P0.0 & P0.1 as TxD & RxD
PINSEL0 = 0x00000005



UART0 Line Control Register -U0LCR

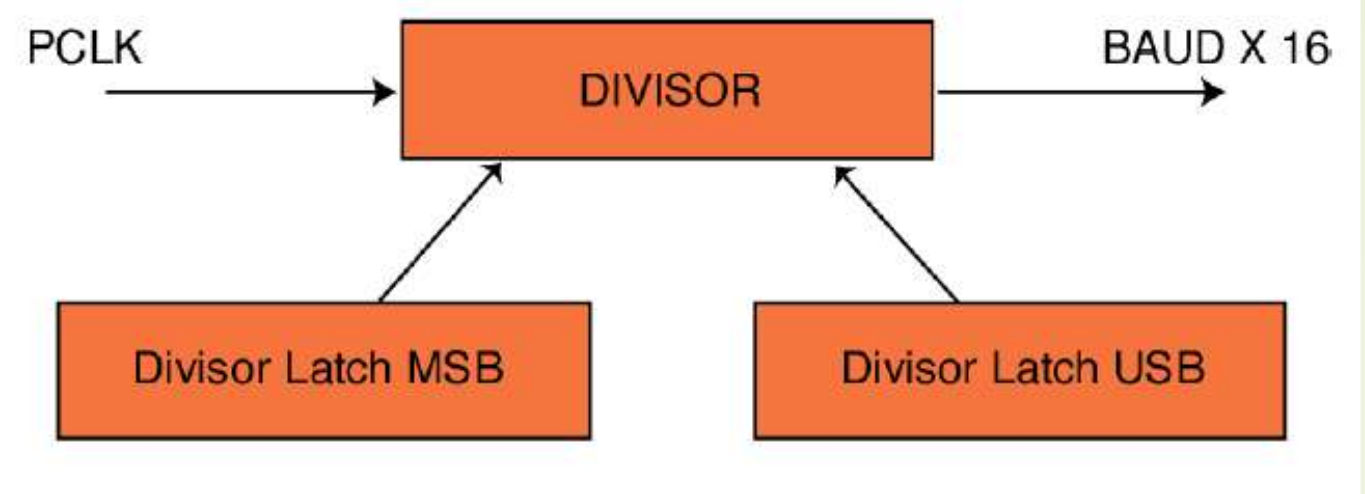
Bit	Symbol	Value	Description	Reset value
1:0	Word Length Select	00	5 bit character length	0
		01	6 bit character length	
		10	7 bit character length	
		11	8 bit character length	
2	Stop Bit Select	0	1 stop bit.	0
		1	2 stop bits (1.5 if U0LCR[1:0]=00).	
3	Parity Enable	0	Disable parity generation and checking.	0
		1	Enable parity generation and checking.	
5:4	Parity Select	00	Odd parity. Number of 1s in the transmitted character and the attached parity bit will be odd.	0
		01	Even Parity. Number of 1s in the transmitted character and the attached parity bit will be even.	
		10	Forced "1" stick parity.	
		11	Forced "0" stick parity.	
6	Break Control	0	Disable break transmission.	0
		1	Enable break transmission. Output pin UART0 TXD is forced to logic 0 when U0LCR[6] is active high.	
7	Divisor Latch Access Bit (DLAB)	0	Disable access to Divisor Latches.	0
		1	Enable access to Divisor Latches.	

UART0 Line Status Register -U0LSR



Bit –0 : Receive Data Ready (RDR)
Bit –5 : Transmit Holding Register Empty (THRE)

Baud Rate Clock



U0DLM U0DLL

```
Divisor  = Pclk/16 x BAUD
```

In our case at 30MHz:

```
Divisor = 30,000,000/16 x 9600 = (approx)
```

DLAB = 1

195 (Or) 0xC3

Baud Rate Calculations



For Pclk = 30MHz

Divisor = Pclk / (Baud x 16)
= 60 x 10^6 / (9600 x 16)
= 391
= 0x01 87
U0DLM = 0x01
U0DLL = 0x87

For Pclk = 60MHz

Divisor = Pclk / (Baud x 16)
= 30 x 10^6 / (9600 x 16)
= 195
= 0x00 C3
U0DLM = 0x00
U0DLL = 0xC3

Divisor Latch

- UART0 Divisor Latch MSB -U0DLM
- UART0 Divisor Latch LSB -U0DLL
- Holds the value used to **divide the clock supplied by the prescaler**in order to produce the baud rate clock.
- The Divisor Latch Access Bit (DLAB) in U0LCR must be **one**in order to access the UART0 Divisor Latches

UART0 Receive Buffer -U0RBR

- The U0RBR is the top byte of the UART0 Rx FIFO
- The Divisor Latch Access Bit (DLAB) in U0LCR must be **zero** in order to access the U0RBR
- The U0RBR is always Read Only

UART0 Transmit Holding Register -U0THR

- The U0THR is the top byte of the UART0 TX FIFO.
- The Divisor Latch Access Bit (DLAB) in U0LCR must be **zero** in order to access theU0THR.
- The U0THR is always Write Only

Algorithm for Initializing UART0

1.Generate Pclk for 30MHz

VPBDIV = 0X00000002;

2. Select UART Function for the Pins

PINSEL0 = 0X00000005;

3. Set UART0 Line Control Register for 8 Bit UART, No Parity ,1 Stop Bit with Divisor Latch Access Bit (DLAB) Enabled.

U0LCR = 0X00000083;

4. Set U0DLL & U0DLM for 9600 Baud.

U0DLL = 0XC3;

U0DLM = 0X00;

5.Clear Divisor Latch Access Bit (DLAB)

U0LCR = 0X00000003;

Initialization Routine

```
void initserial(void)

{
VPBDIV = 0X02;
PINSEL0 =0X00000005;
U0LCR = 0X00000083;
U0DLL = 0XC3;
U0DLM = 0X00;
U0LCR = 0X00000003;
}
```

Algorithm for Transmitting Data using UART0

- Move the data to be transmitted to UART0 Transmit Holding Register.

```
U0THR = mydata;
```

Wait until Transmit Holding Register Empty is set.

```
while((U0LSR & 0x20)==0);
```

Send Serial Routine

```
void send serial(unsigned char serdata)

{
U0THR = serdata;
while((U0LSR & 0x20)==0);
}
```

Algorithm for Receiving Data using UART0

Wait until Receive data Ready Flag is set.

```
while((U0LSR & 0x01)==0);
```

- Move the received data from UART0 Receive Buffer Register .

```
temp = U0RBR ;
```

Receive Serial Routine

```
unsigned char Receive Serial(void)
{
unsigned char my data ;
while((U0LSR & 0x01)==0);
My data = U0RBR ;
return my data;
}
```

Algorithm for Receiving Data using UART0

Wait until Receive data Ready Flag is set.

```
while((U0LSR & 0x01)==0);
```

- Move the received data from UART0 Receive Buffer Register .

```
temp = U0RBR ;
```

Receive Serial Routine

```
unsigned char Receive Serial(void)
{
unsigned char my data ;
while((U0LSR & 0x01)==0);
My data = U0RBR ;
return my data;
}
```

UART.h

- **void**initserial**(void);**
- **void**sendserial**(unsignedchar**serdata**);**
- **unsignedchar** receiveserial**(void);**

initserial

```
voidinitserial(void)
{
PINSEL0 =0X05;
// P0.0 = TxD , P0.1 = RxD
U0LCR =0X83;
// DLAB(7) = 1 , 8 Bit UART (1,0) = 1,
U0DLL =0X87;
// = 60 x 10^6 / (9600 x 16) = 391 = 0x0187
U0DLM =0X01;// 9600 Baud
U0LCR =0X03;// DLAB(7) = 0,8 Bit UART (1,0) = 1,
}
```

sendserial

```
void sendserial(unsigned char serdata)
{
U0THR =serdata;
// Move data to be transmitted to U0THR
while((U0LSR &0x20)==0);
// wait until THR Empty Flag is set.
}
```

receiveserial

```
unsigned char receiveserial(void)
{
unsigned char mydata ;
while((U0LSR &0x01)==0);
// wait until RBR Full Flag is set.
mydata =U0RBR ;
// Move data Received to U0THR
return mydata;
}
```